

PROMPTOS (PROMPT ENGINEERING OPERATING SYSTEM)

Comprehensive Technical & Architectural Project Report

Executive Summary

PromptOS is an enterprise-grade Prompt Engineering Operating System designed to bridge the gap between vague user ideas and production-grade, model-tailored AI prompts. By combining local/cloud AI consultation, domain-driven prompt strategies, retrieval-augmented generation (RAG) for corporate guidelines, static linting, model-specific formatting engines, and multi-model benchmarking, PromptOS systematically optimizes prompts across 15+ LLM providers and specialized AI coding agents.

1. Introduction

1.1 Context and Problem Statement

As Large Language Models (LLMs) proliferate across enterprise applications, organizations face severe challenges:

Prompt Quality Variance: Casual or under-specified prompts yield hallucinated, incomplete, or non-compliant outputs from LLMs.

Provider-Specific Best Practices: Prompting techniques that excel on Anthropic models (e.g., XML tags, literal scope adherence) differ significantly from OpenAI (e.g., Markdown system prompts, JSON mode instructions) or coding agents (e.g., Cursor, GitHub Copilot).

Lack of Enterprise Compliance: User prompts often ignore organizational coding standards, security constraints, and compliance rules.

Model Lock-In: Re-formatting prompts for different providers or local models requires manual effort and domain knowledge.

1.2 Purpose of PromptOS

PromptOS automates and standardizes the prompt lifecycle. It transforms unstructured user intentions into highly structured, provider-optimized system and user instructions, enforcing enterprise guardrails, applying advanced prompting strategies (Chain-of-Thought, ReAct, Few-Shot), and evaluating execution metrics across multiple target models.

2. Project Full Description

PromptOS is built as a decoupled, microservices-ready full-stack platform consisting of a high-performance FastAPI (Python) backend and an interactive Next.js 15 (TypeScript) web application.

2.1 Core Capabilities

Interactive AI Consultation: Utilizes a fast local LLM (qwen2.5:1.5b via Ollama) or cloud fallback to adopt specialized expert personas (e.g., Senior Software Architect, CMO, Financial Advisor) and ask 2–3 surgical clarifying questions before prompt compilation.

Vector-Driven Guideline RAG: Leverages Qdrant Vector Database and FastEmbed (BAAI/bge-small-en-v1.5) to search and inject relevant corporate compliance rules and best practices directly into the prompt context.

Provider & Model Compilation Engine: Features an extensible plugin architecture supporting 15+ providers (Anthropic, OpenAI, DeepSeek, Google Gemini/Gemma, Meta Llama, xAI Grok, Moonshot Kimi, Perplexity, Microsoft Copilot, Cursor, GitHub Copilot, Codex, Blackbox AI, and Local Qwen). Each plugin applies provider-native formatting rules (e.g., XML tag nesting for Claude, Markdown sectioning for GPT, or agentic instructions for Cursor).

Domain Strategy Engine: Dynamically assigns optimal prompting frameworks—such as Chain-of-Thought (CoT) for programming/math, ReAct for research/tool tasks, or Few-Shot when examples are present.

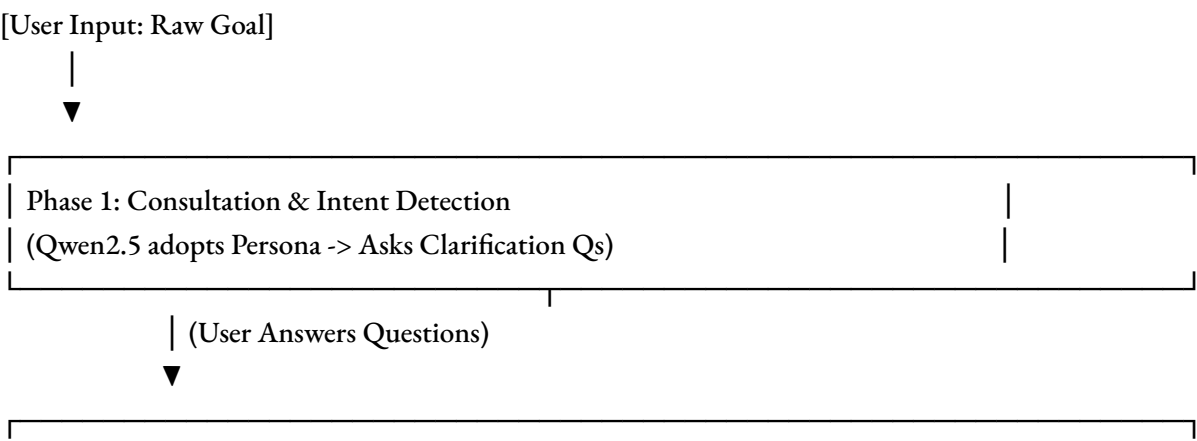
Static Prompt Linter: Evaluates prompt specifications against quality rules (e.g., Role Assignment Rule, Ambiguous Format Rule) and calculates quality scores.

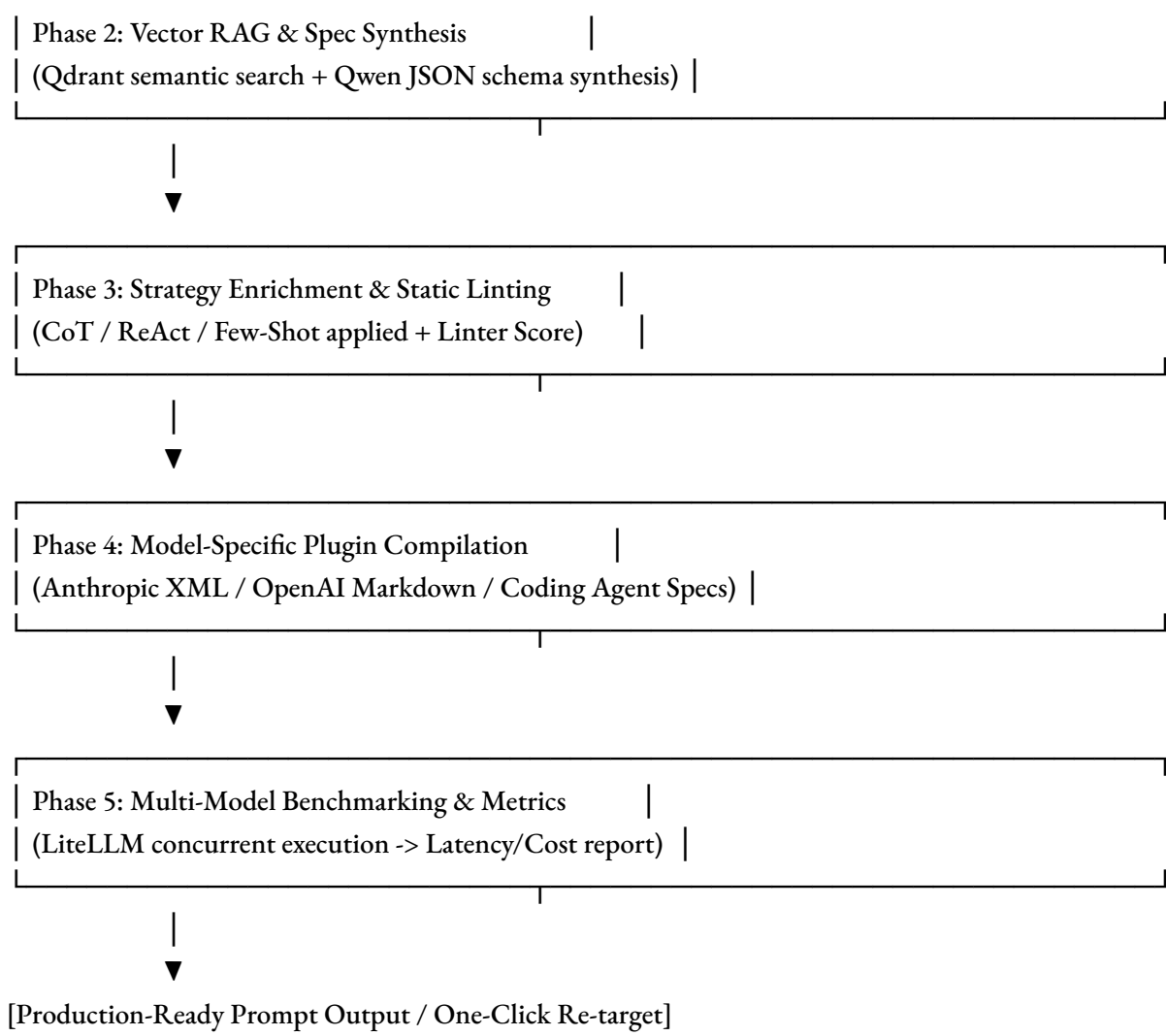
Multi-Model Benchmarking & Cost Metrics: Uses LiteLLM to run compiled prompts concurrently across target models, calculating latency (ms), token usage, and total cost (USD) to identify optimal model choices.

Iterative Refinement & Instant Provider Swapping: Allows users to instantly recompile their specification for any target model with a single click, or supply refined criteria in a continuous feedback loop.

3. How It Works

PromptOS follows a structured 6-phase compilation and execution pipeline:





Step-by-Step Workflow

User Goal Entry: The user selects a target model (e.g., Claude Sonnet 5, DeepSeek Reasoner, Cursor AI Agent) and enters a high-level prompt goal.

Phase 1 Pass (Consultation): If interactive mode is enabled, PromptOS dispatches the prompt to local Qwen (qwen2.5:1.5b). Qwen adopts the relevant professional role and outputs structured JSON questions to clarify missing constraints, schemas, or target context.

Phase 2 Pass (RAG & Synthesis): The backend queries Qdrant to retrieve matching enterprise guidelines via FastEmbed embeddings (BAAI/bge-small-en-v1.5). Qwen then synthesizes the original prompt, user answers, and retrieved guidelines into a clean, normalized TaskSpecification schema.

Phase 3 Pass (Strategy & Linting): The StrategyEngine auto-selects prompting tactics (CoT, ReAct, Few-Shot, Direct) while the PromptLinter validates system prompt compliance.

Phase 4 Pass (Plugin Formatting): The selected provider plugin converts the TaskSpecification into native ChatML / system-user message structures with provider-specific formatting (e.g., Claude XML tags or Cursor system directives).

Phase 5 Pass (Benchmarking & Output): The compiled output is rendered in the UI with copy options, provider swapping capability, and optional multi-model benchmarking.

4. Requirements and Specifications

4.1 Functional Requirements

- FR-1: Accept unstructured prompts in interactive or direct compilation modes.
- FR-2: Dynamically adopt expert personas and generate context-aware clarifying questions in JSON.
- FR-3: Perform real-time vector similarity search over corporate guidelines using cosine distance.
- FR-4: Support plug-and-play provider adapters via a unified ProviderPlugin interface.
- FR-5: Sanitize and validate LLM synthesis output to prevent Pydantic schema validation failures.
- FR-6: Support instant provider target switching without losing gathered intent and context.
- FR-7: Benchmark execution performance across multiple LLMs concurrently using LiteLLM.

4.2 Non-Functional Requirements

- NFR-1 Latency: Consultation and synthesis requests using local Qwen should return in under 3 seconds on hardware with GPU acceleration or 5–10 seconds on CPU.
- NFR-2 Scalability: Asynchronous FastAPI endpoints (async/await) handling concurrent compilation requests.
- NFR-3 Resilience: Automatic fallback to raw specification structure if Qwen synthesis or local Ollama service fails.
- NFR-4 Observability: OpenTelemetry tracing integrated with Jaeger for monitoring end-to-end request pipelines.
- NFR-5 Extensibility: Developers must be able to register new LLM plugins in under 20 lines of Python code.

4.3 Data Specifications

The primary data transfer object is the TaskSpecification model:

python

```
class TaskSpecification(BaseModel):
    category: str                # e.g., programming, research, writing
    primary_objective: str        # Core task description
    role_persona: Optional[str] = None    # Adopted expert role
    context_data: List[str]       # Retrived enterprise RAG guidelines
    constraints: List[str]        # Operating boundary conditions
    output_format: Optional[Dict[str, str]]    # Structured output schema (e.g., JSON)
...3 lines truncated. Use the buttons above to view or insert the full code.
```

5. Technologies Has Been Used

5.1 Backend Technology Stack

Language: Python 3.11+
Framework: FastAPI (ASGI web server for high-concurrency API endpoints)
Data Validation: Pydantic v2 (Strict schema definition and payload sanitization)
HTTP Client: HTTPX (Asynchronous HTTP requests for Ollama and external APIs)
Vector Embeddings: FastEmbed (BAAI/bge-small-en-v1.5, 384-dimensional local embeddings)
LLM Abstraction: LiteLLM (Unified model execution, token tracking, and cost calculation)
ORM & Database Client: SQLAlchemy / asyncpg for PostgreSQL operations

5.2 Frontend Technology Stack

Framework: Next.js 15+ (App Router, Server & Client Components)
UI Library: React 19
Language: TypeScript (Strict type safety)
Styling: Tailwind CSS v3/v4 & Shadcn UI / Radix Primitives
State Management: Zustand (Global state management for compilation steps and prompt state)
Icons: Lucide React

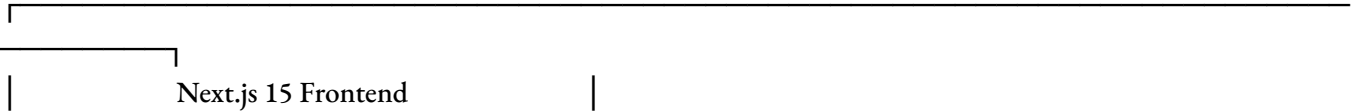
5.3 Infrastructure & Databases

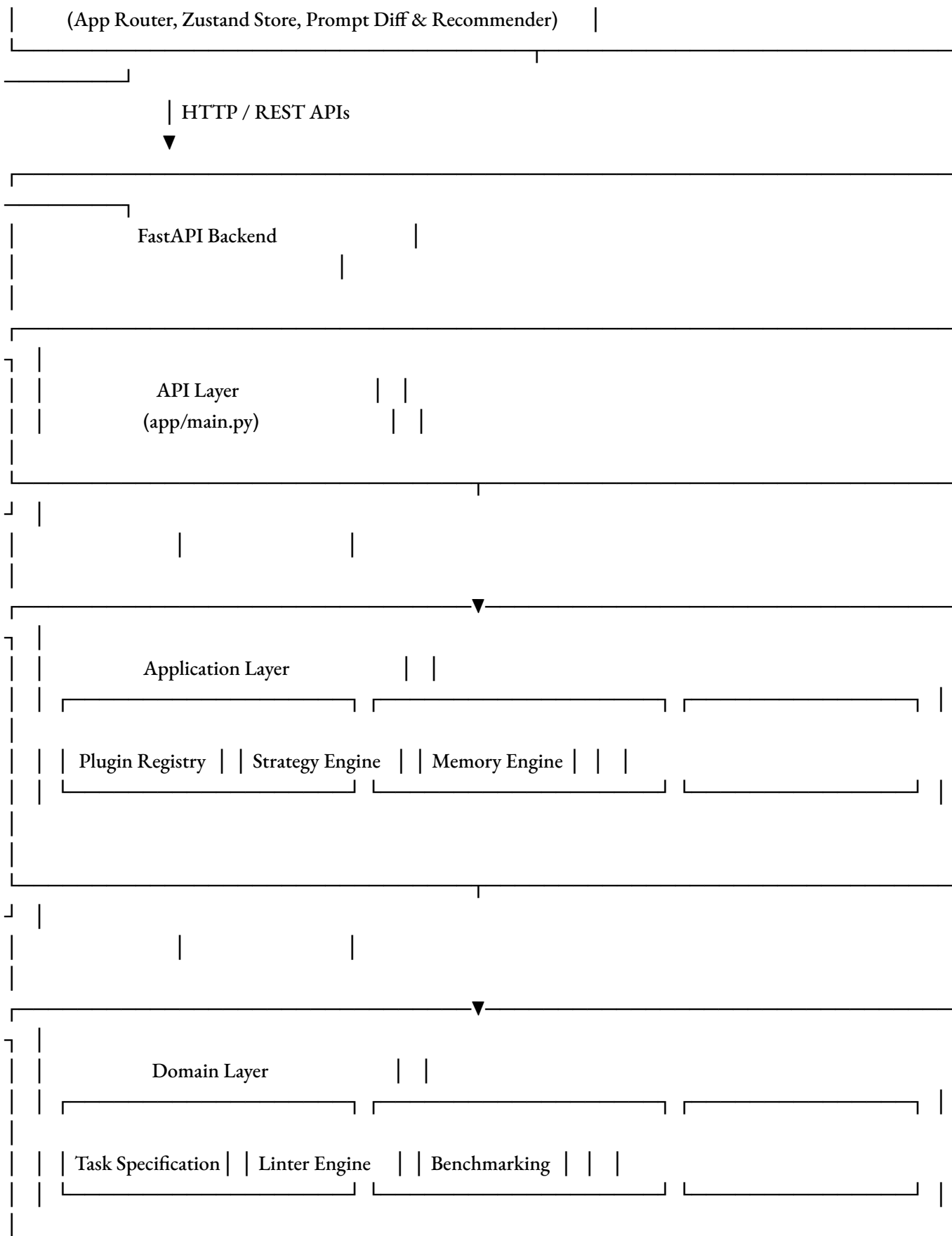
Containerization: Docker & Docker Compose v3.8
Vector Database: Qdrant (Vector search engine running on port 6333/6335)
Relational Database: PostgreSQL 15 Alpine (Port 5432/5433) for execution logs and state persistence
Caching & Queueing: Redis 7 Alpine (Port 6379/6380)
Observability: Jaeger Tracing (Port 16686 UI, Port 4317 OTLP gRPC)
Local Inference Engine: Ollama running qwen2.5:1.5b model

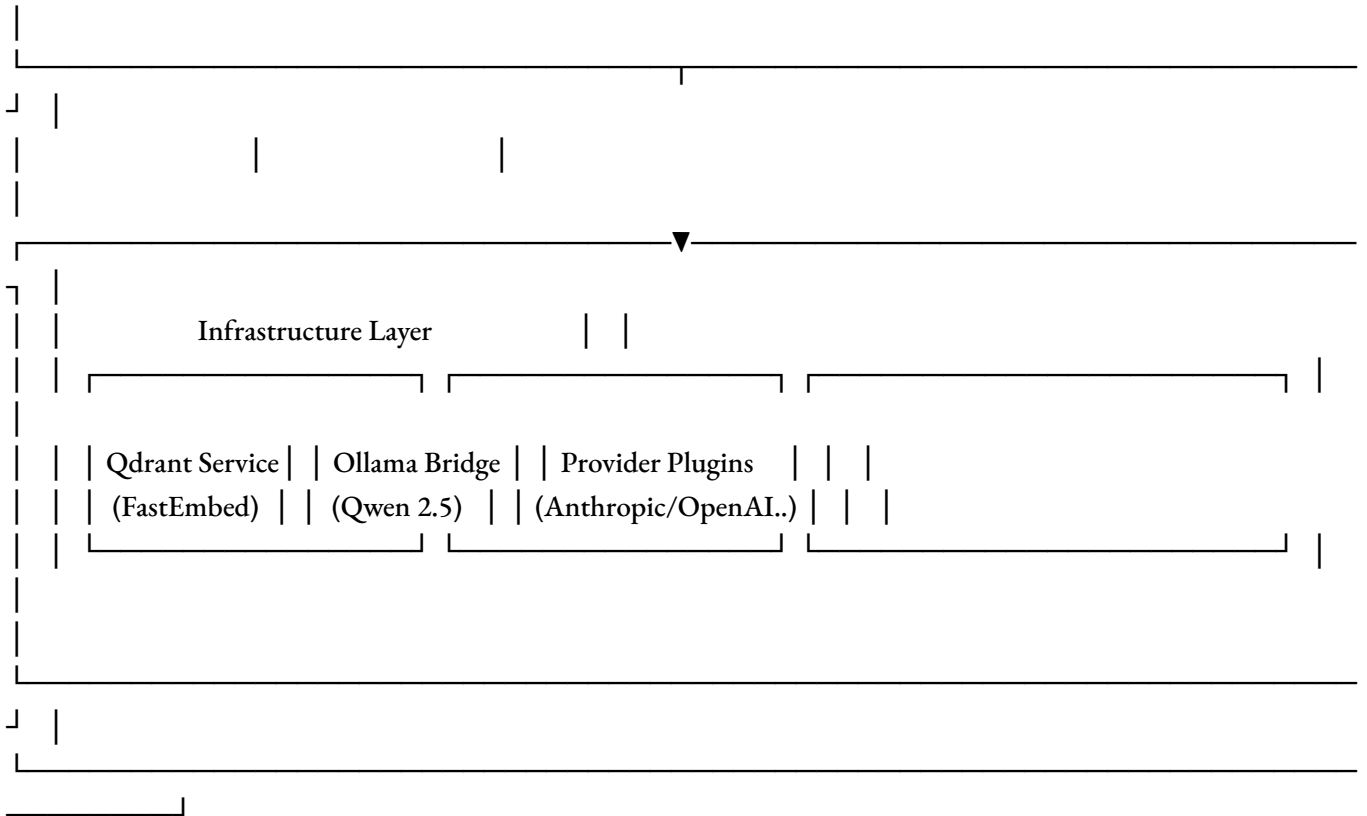
6. Project Architecture

6.1 Architectural Overview

PromptOS follows Domain-Driven Design (DDD) and Clean Architecture principles, ensuring strict segregation between domain models, application logic, infrastructure adapters, and API interfaces.

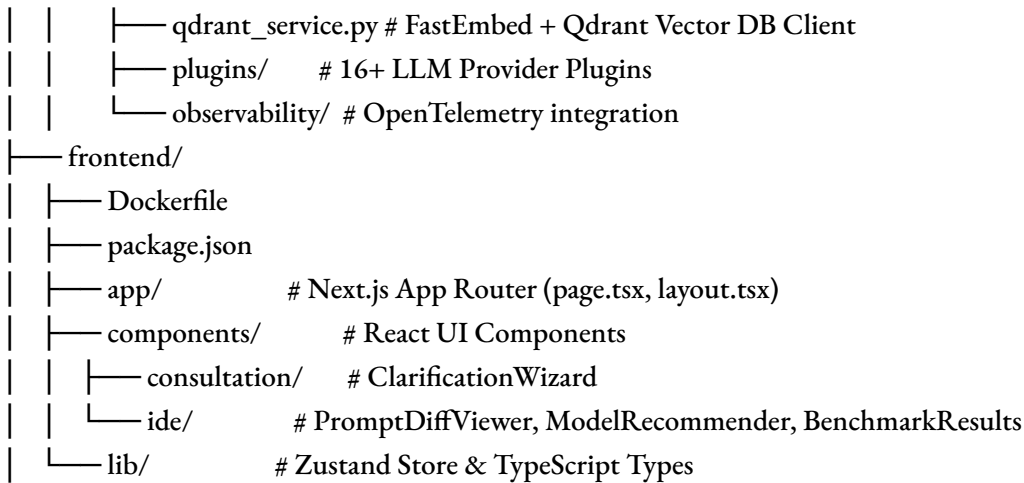






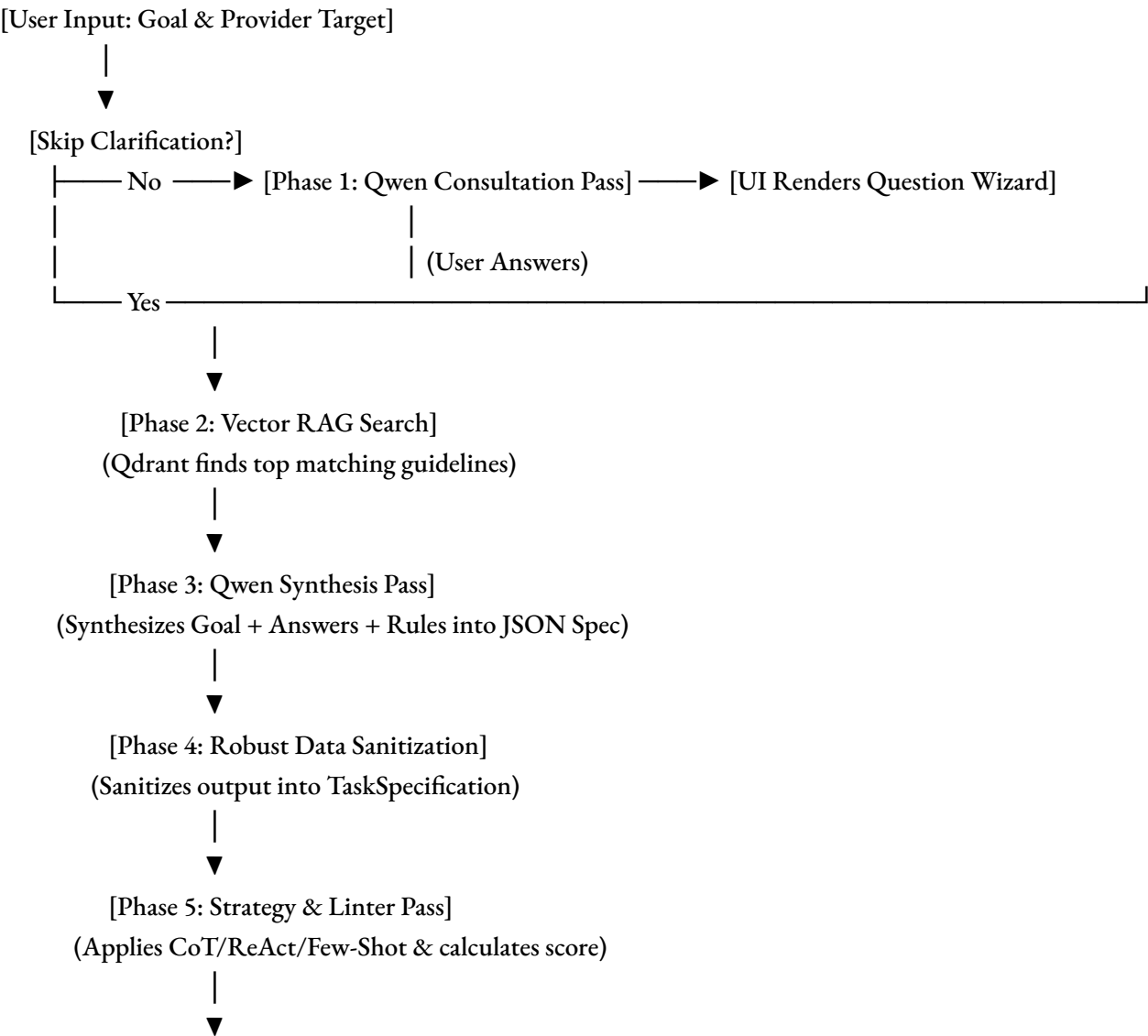
6.2 Component Directory Structure

```
promptos/
├── docker-compose.yml    # Infrastructure orchestration (6 container services)
├── .env                  # Centralized environment configurations
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
│       ├── main.py      # FastAPI entrypoint & endpoint orchestration
│       ├── domain/      # Entities, Specifications & Interfaces
│       │   ├── models/  # TaskSpecification, TaskCategory, PromptStrategy
│       │   ├── linter/  # Linter rules (Role Rule, Ambiguous Format Rule)
│       │   ├── benchmarking/ # Benchmark metrics models
│       │   └── recommendation/ # Model ranking models
│       ├── application/  # Application Use Cases
│       ├── plugin_registry.py # Provider Plugin Registry
│       ├── compiler/     # Strategy Engine
│       ├── intent/       # Intent Analyzer & Requirement Gatherer
│       ├── knowledge/    # Knowledge enrichment engine
│       ├── memory/       # Historical execution memory
│       └── infrastructure/ # External Services & Adapters
```



7. Project Full Pipeline

The full execution pipeline spans both interactive frontend user flows and backend processing routines:



[Phase 6: Provider Plugin Compilation]
(Formats into Anthropic XML, OpenAI Markdown, etc.)



[Rendered Output & Post-Generation Actions]
(Re-target provider / Refine loop / Benchmark)

Detailed Pipeline Mechanics

Request Reception: Endpoint `/api/v1/compile` receives `CompileRequest` with `user_prompt`, `target_provider`, `skip_clarification`, and optional `resolved_clarifications`.

Qwen Pass 1 (Consultant): `ask_qwen_for_clarifications()` sends client goals to local Ollama (qwen2.5:1.5b). Qwen adopts an industry persona and returns clarification questions in JSON format.

Guideline RAG: `qdrant_db.search_guidelines()` converts the prompt into a 384-d vector via FastEmbed and queries Qdrant for top-3 relevant enterprise rules (score threshold > 0.5).

Qwen Pass 2 (Synthesizer): `synthesize_prompt_details()` combines user input, clarification answers, and RAG context into structured sections (Primary Objective, Context, Constraints, Examples).

Data Sanitization: `sanitize_rich_spec()` normalizes dictionary types, flattens nested arrays, and formats examples into strict user/assistant pairs so Pydantic parsing never fails.

Compiler & Plugin Execution: `registry.get(target_provider)` retrieves the provider adapter and compiles the specification into ChatML formatted prompt strings.

8. Provider Plugin Ecosystem

PromptOS includes built-in support for a vast ecosystem of cloud LLMs, open-source models, and AI coding environments:

Provider Category	Provider Key	Supported Models / Engines	Special Compilation Features
Anthropic	anthropic	Claude Sonnet 5, Opus 5, Opus 4.8	Structured <code><enterprise_context></code> , <code><constraints></code> , <code><examples></code> , and <code><objective></code> XML tag nesting; cache-optimized layout.
OpenAI	openai_gpt4, openai	o3-mini, GPT-4 / Omni, GPT-4.5	Structured Markdown headers, explicit role system prompts, native JSON mode instructions.
DeepSeek	deepseek	DeepSeek Reasoner (R1), DeepSeek V3	Forces step-by-step reasoning triggers and chain-of-thought scratchpad blocks.
Google	google_gemini, google	Gemini 1.5 Pro, Flash, Gemma 2	Multi-modal context formatting and system instruction wrapping.
Meta	meta	Llama 3.3 (70B) `<	
xAI	xai	Grok-1	Direct truth-seeking prompt constraints and analytical formatting.
Moonshot AI	moonshot	Kimi K2.5	Multi-turn long-context retention structure.
Perplexity	perplexity	Sonar	Enforces real-time search citation requirements and structured answer formats.

Microsoft	microsoft_copilot	Microsoft Copilot (M365)	Enterprise M365 context integration, compliance-friendly language rules.
Coding Agents	cursor, github_copilot, openai_codex, blackbox	Cursor AI, GitHub Copilot, Codex, Blackbox	Generates .cursorsrules / system rules with strict repo file scoping, code formatting, and test-driven output constraints.
Local Models	qwen_local	Local Qwen via Ollama	Lightweight ChatML formatting optimized for local small language models.
Cloud Gateway	openrouter	OpenRouter Unified API	Global multi-model gateway support.

9. Prerequisites, What Need to Run, and How to Run

9.1 Hardware and System Requirements

Operating System: Windows 10/11 (PowerShell/WSL2), macOS, or Linux (Ubuntu 20.04+).

RAM: Minimum 8 GB (16 GB recommended if running local Ollama model).

Disk Space: 10 GB free space (for Docker images and Qwen model weights).

Software Dependencies:

Docker Desktop (v20.10 or higher) with Docker Compose v2.

Node.js (v18.x or higher) (only if running frontend standalone).

Python (v3.11 or higher) (only if running backend standalone).

Ollama with qwen2.5:1.5b model pulled:

```
powershell
```

```
ollama run qwen2.5:1.5b
```

9.2 Environment Configuration (.env)

Create a .env file in the root directory (or use the provided .env template):

```
env
```

```
# Core Application Settings
```

```
USE_CLOUD_MODELS=false
```

```
PROMPTOS_APP_NAME=PromptOS
```

```
PROMPTOS_SITE_URL=http://localhost:3001
```

```
# Database & Infrastructure URLs
```

```
DATABASE_URL=postgresql://promptos:secret@postgres:5432/promptos
```

```
QDRANT_URL=http://qdrant:6333
```

```
REDIS_URL=redis://redis:6379/0
```

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317
```

```
# Provider API Keys (Optional - Required for live API benchmarking)
OPENAI_API_KEY=your_openai_api_key_here
ANTHROPIC_API_KEY=your_anthropic_api_key_here
OPENROUTER_API_KEY=your_openrouter_api_key_here
...3 lines truncated. Use the buttons above to view or insert the full code.
```

9.3 How to Run via Docker Compose (Recommended)

The entire application stack (Backend, Frontend, PostgreSQL, Qdrant, Redis, Jaeger) can be launched with a single command.

Step 1: Clone and Navigate to Repository

powershell

```
cd c:\Users\salma\OneDrive\Desktop\promptos
```

Step 2: Ensure Ollama is Running locally with Qwen

powershell

```
ollama pull qwen2.5:1.5b
```

Step 3: Build and Start Containers

powershell

```
docker-compose up --build -d
```

Step 4: Verify Container Health

powershell

```
docker-compose ps
```

Step 5: Access the Applications

PromptOS Web Dashboard: <http://localhost:3001>

FastAPI Backend Documentation: <http://localhost:8002/docs>

Jaeger Observability UI: <http://localhost:16686>

Qdrant Dashboard: <http://localhost:6335/dashboard>

9.4 How to Run Standalone (Development Mode)

If you prefer to run the backend and frontend locally for development without Docker:

Running Backend Standalone

Open terminal in ./backend:

powershell

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1 # On Windows PowerShell
pip install -r requirements.txt
Start infrastructure dependencies (Qdrant & Postgres) via Docker or local installation:
powershell
```

```
docker-compose up postgres qdrant redis -d
Launch FastAPI backend:
powershell
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8002 --reload
Running Frontend Standalone
Open terminal in ./frontend:
powershell
```

```
cd frontend
npm install
Launch Next.js dev server:
powershell
```

```
npm run dev
Open browser at http://localhost:3000.
```

10. Verification and Testing

PromptOS includes automated test suites covering plugin registration, intent analysis, requirement gathering, and model recommendation logic.

To run backend tests:

```
powershell
```

```
cd backend
pytest tests/
To run end-to-end frontend compilation tests:
```

```
powershell
```

```
cd frontend
npx playwright test
```

11. Security, Observability, and Governance

11.1 Enterprise Governance & RAG Ingestion

PromptOS enforces company-wide prompting guidelines by embedding rules in Qdrant. Enterprise administrators can upload internal documentation (e.g., security guidelines, coding standards, brand voice) via `QdrantService.upload_guideline()`. Every user prompt is automatically evaluated against these guidelines before output generation.

11.2 Observability with Jaeger & OpenTelemetry

11.2 Observability with Jaeger & OpenTelemetry

Every prompt compilation request creates a distributed trace across the backend:

Span 1: Intent Detection & Qwen Consultation Latency

Span 2: FastEmbed Vector Generation & Qdrant Search Latency

Span 3: Spec Synthesis & Data Normalization

Span 4: Provider Plugin Formatting

Traces can be visualized in real-time in the Jaeger UI at <http://localhost:16686>.

12. Conclusion

PromptOS delivers a complete, modular solution to the prompt engineering challenge in modern enterprise AI deployments. By transforming ambiguous user prompts into structured, strategy-enriched, and provider-optimized prompts, PromptOS significantly elevates output quality, ensures strict compliance with organizational guidelines, and drastically cuts iteration time across 15+ major LLM platforms.

With its clean architecture, microservices infrastructure (FastAPI, Next.js, Qdrant, PostgreSQL, Redis, Jaeger), extensible plugin system, and local inference capabilities (Ollama Qwen), PromptOS is fully prepared for enterprise deployment and continuous scaling.

Report compiled for PromptOS Architecture & Engineering Team.